Pipeline Comparison — v3 vs. New Platform

What carries forward from the delivered POC, what has been rebuilt, and why

Prepared for Digital Direction
April 2026

Context. The **previous platform (v3)** validated the 60-field inventory approach on real client data across Windstream, Peerless, AT&T, and Spectrum — hitting **98.59%** on the largest Windstream account and proving the extract-merge-QA workflow end-to-end. The **new platform** keeps the shape of that pipeline and its 60-field schema, but rebuilds the core extraction, merge, and storage layers so that **new carriers onboard through configuration alone**, scanned documents are handled natively, and every human correction improves future extractions.

> **In one line.** v3 proved the workflow with hand-coded carrier logic. The new platform generalises that workflow into a configuration-driven, LLM-first pipeline with a persistent audit trail and a self-healing feedback loop — so scaling to new carriers and document formats stops being an engineering project.

AT A GLANCE

DIMENSION	PREVIOUS PLATFORM (V3)	NEW PLATFORM
Core approach	Deterministic Python parsers + merge engines **hand-coded per carrier**. LLM used only for optional file classification and OCR.	**LLM-first extraction** (Gemini 2.5 Flash / Pro, adaptive) driven by carrier-specific prompts and YAML domain knowledge. Claude used narrowly for merge-conflict resolution.
New-carrier onboarding	YAML config **plus** per-carrier Python: parser modules (one per file type), a merge engine, a carrier plugin class, registration. Typically days of engineering per carrier.	**Configuration only**: a carrier folder with `carrier.yaml`, `formats/*.yaml`, `prompts/*.md`, and `domain_knowledge/*.yaml`. Target: <1 day to first extraction, no code change.
Scanned PDFs / OCR	Separate OCR code path (per-carrier and generic). Peerless coverage capped by OCR gaps (~50% of rows blocked in the POC).	Multimodal Gemini 2.5 Pro as a **native branch of the main flow**; no separate OCR subsystem. Scanned PDFs are chunked and extracted with the same prompts and schema.

DIMENSION	PREVIOUS PLATFORM (V3)	NEW PLATFORM
Classification	File-name regex + directory patterns; optional LLM classifier with cached manifest.	**Three-stage**: filename → first-page content signals → LLM fallback on disagreement. Post-extraction check re-classifies if extracted `carrier_name` contradicts the initial call.
Merge logic	Hand-written Python merge engine per carrier (~300–500 lines each), plus YAML source-priority chains.	Config-driven **field priority matrix** (carrier-overridable) for the common case; Claude invoked only where two high-confidence sources genuinely conflict.
QA / Validation	30 hard-coded QA rules (some fire incorrectly on Peerless by design).	Cross-field math + format checks + confidence scoring, plus a dedicated **Compliance stage** (rate-mismatch, expired contract, M2M inconsistency, term-date mismatch, no-contract).
Feedback loop	Corrections require a pipeline re-run; no persistent feedback back into extraction.	Human corrections persist and are **injected as "known corrections" into future extraction prompts**, alongside few-shot examples and domain-knowledge updates.
Persistence	SQLite as transient staging; Excel is the permanent record.	**PostgreSQL + pgvector** as the system of record: uploads, documents, extraction runs, extracted rows, corrections, with soft-delete / retention.
Throughput	Per-account processing, primarily sequential within a run.	Section-level parallelism with up to **200 concurrent Gemini calls**; per-file extractions cached for crash-safe resume.
Frontend	React + Material-UI; live WebSocket status, filterable results table, Excel download.	Next.js + shadcn/ui; Dashboard, Upload, Results, Review, Analytics, Bin, Settings. Review page designed to show the source document beside extracted fields with confidence-based triage.
Output	Multi-sheet Excel as the deliverable; source attribution stays in Python and is not exposed in the workbook.	60-field rows stored with **per-row** `source_documents`, `compliance_flags`, and per-field confidence. Excel export remains available; the audit trail lives in the database.

PIPELINE SHAPE — SIDE BY SIDE

01 **Discover** Regex / aliases	02 **Parse** Per-carrier Python
03 **Merge** Hand-coded engine	04 **QA** 30 rules
05 **Export** Excel + SQLite	

01 **Classify** 3-stage + re-check	02 **Parse** Docling · pdfplumber · pandas · multimodal
03 **Extract** Gemini 2.5 + prompts	04 **Merge** Priority matrix + Claude
05 **Validate + Comply** Math · format · compliance	

The **shape** is preserved so the workflow remains familiar: documents in, 60-field inventory out, reviewer confirms. What changed is *how each stage is implemented* and *where carrier-specific knowledge lives*.

STAGE BY STAGE — WHAT ACTUALLY CHANGED

STAGE	PREVIOUS PLATFORM (V3)
Classify / Discover	Directory scan with carrier YAML regexes and aliases (`invoices` , `reports` , `contracts` , `csrs`). Optional Gemini Flash classifier with a cached manifest for files that don't match patterns.

STAGE	PREVIOUS PLATFORM (V3)
Parse	One module per carrier per file type — e.g. `src/parsers/windstream/{invoice,csr,mrc_report,carrier_report,contract,cams_report}.py` plus Peerless/AT&T/Spectrum equivalents. Separate OCR path under `src/parsers/ocr/`.
Extract	No LLM in the extraction path. Each parser emits typed dataclasses assembled by hand from regex, pdfplumber layout, and pandas. Extending to a new format means writing more Python.
Merge	A separate merge engine per carrier (`src/merge/{windstream, peerless, att, spectrum, ...}.py`). Shared `source_resolver.py` applies YAML priority chains per field. S/C hierarchy logic is

STAGE	PREVIOUS PLATFORM (V3)
	carrier-specific and lives in Python.
Validate	30 QA rules implemented as Python functions in `src/rules/qa_checklist.py`, plus per-field confidence in `src/rules/confidence.py`. Some rules fire by design on Peerless flat-S output and need manual dismissal.
Compliance new stage	—

STAGE	PREVIOUS PLATFORM (V3)

ADDING A NEW CARRIER — THEN VS. NOW

PREVIOUS PLATFORM

1. Write `src/config/{carrier}.yaml` (directories, regexes, priorities)
2. Create `src/parsers/{carrier}/` — one module per file type
3. Create `src/merge/{carrier}.py` — merge engine with S/C logic
4. Create `src/carriers/{carrier}.py` — `CarrierPlugin` class
5. Register in `src/carriers/__init__.py`
6. Add routing in `main.py` and gold accounts in `backend/config.py`

Result: days of engineering, code review, and testing for each carrier or major format variant.

NEW PLATFORM

1. Create `configs/carriers/{name}/carrier.yaml` — aliases, filename patterns, account patterns, first-page signals, merge rules
2. Add `formats/*.yaml` — one per format variant (signature, chunking, field rules, examples)
3. Add `prompts/*.md` — extraction prompts per document type
4. Populate `domain_knowledge/*.yaml` — USOC codes, field codes, service types (optional)
5. Restart the server — `ConfigStore` loads on startup

Result: no Python changes. Target of <1 day to first extraction, <1 week to 80%+ accuracy.

CONFIDENCE, AUDIT TRAIL, AND THE FEEDBACK LOOP

CAPABILITY	PREVIOUS PLATFORM (V3)	NEW PLATFORM
Per-field confidence	Assigned by rule in `src/rules/confidence.py` (HIGH / MEDIUM / LOW buckets by field category). Lives in Python, not persisted alongside the row.	Assigned per field by the validator (HIGH / MEDIUM / LOW / MISSING) and stored so the review UI can **auto-accept HIGH, pre-fill MEDIUM, flag LOW**.
Source attribution	`ResolvedField` tracks winning source and alternatives, with conflict logs in	Every row carries a `source_documents` JSONB list of contributing documents; per-

CAPABILITY	PREVIOUS PLATFORM (V3)	NEW PLATFORM
	Python — but the final Excel does not expose per-field source.	field source tracking is designed into the schema and visible to reviewers.
Feedback loop	No persistent correction store. Improving a miss means re-running with updated YAML or updated code.	Corrections are persisted and re-injected on the next run via three channels: • Few-shot examples in the extraction prompt • YAML domain-knowledge updates (new USOCs, new field mappings) • Known-correction hints ("do not extract X as *wrong*; correct is *right*")
Crash-safety / resume	OCR cache and classification manifest reused across runs. A mid-run failure typically restarts the account.	Per-file extractions cached to `/data/cache/extractions/`; a resumed run reuses completed work and only re-does what's missing.

INFRASTRUCTURE & SCALE

AREA	PREVIOUS PLATFORM (V3)	NEW PLATFORM
Backend	FastAPI, JWT auth, WebSocket streaming for live status. SQLite (`inventory.db`) for transient staging.	FastAPI with SQLAlchemy async, **PostgreSQL + pgvector** as system of record. Uploads / documents / extraction runs / extracted rows / corrections modeled explicitly; soft-delete with bin retention.
Frontend	React + TypeScript + Material-UI. Pages: Dashboard, ProjectDetail, Checklist, Projects. Column-toggle results table, Excel download.	Next.js + Tailwind + shadcn/ui. Pages: Dashboard, Upload, Results, Review, Analytics, Bin, Settings. Review UI aligns extracted fields with the source document for reviewer triage.
Models	Gemini 2.5 Flash for optional classifier and OCR; Anthropic Claude configured as fallback.	Gemini 2.5 Flash / Pro (adaptive by format familiarity), Pro multimodal for scanned PDFs. Claude for merge-conflict resolution only.
Parallelism	Per-account sequential processing inside each pipeline run.	Section-level batches with a semaphore allowing up to **200 concurrent Gemini calls**. A 472-page enterprise bill extracts in the order of seconds.
Deployability	Single FastAPI process serving API + static frontend; SQLite on disk. Cloud Build config present.	Docker Compose for Postgres + Redis; backend and frontend run independently. Caches live on disk under `/data/`.

WHAT CARRIES FORWARD — AND WHAT DOESN'T

KEPT UNCHANGED

- **The 60-field schema** (DD2 info, location, carrier, service, component, circuit speeds, Z-location, contract terms)
- **S / C row typing** — Service summary vs. Component line item
- **Human-in-the-loop review** as the delivery model
- **Carrier configs in YAML** as the unit of extension
- **Target accuracy bands** — >98% structured, >90% semi-structured, >80% fuzzy, >75% contract
- **Excel export** for handoff to downstream workflows

REBUILT NEW PLATFORM

- Extraction moved from hand-coded parsers to **LLM extraction with carrier prompts**
- Carrier onboarding moved from **code + YAML** to **YAML + prompts only**
- OCR folded into the main flow via **multimodal extraction**
- Storage moved from **SQLite + Excel** to **PostgreSQL + pgvector**
- Classification hardened with **three-stage agreement + post-extraction re-check**
- QA extended with a dedicated **Compliance stage**
- Throughput moved from **sequential** to **200-way parallel** with crash-safe caching
- A **persistent correction loop** now improves extractions between runs

> **Why this matters for the client.** The v3 platform is a validated POC on a fixed carrier set. The new platform is a foundation for expanding carriers and clients without re-opening the codebase: a new carrier is a *configuration drop*, a new format variant is a *format YAML*, and every correction a reviewer makes makes the next run better.

APPENDIX — FILE-LEVEL MAPPING (FOR ENGINEERS)

CONCERN	PREVIOUS (`DIGITAL-DIRECTION`)	NEW (`PLATFORM/`
Pipeline orchestration	`main.py` , `backend/services/pipeline_runner.py`	`backend/pipeline`
Classification	`src/pipeline/discovery.py` , `src/pipeline/llm_classifier.py`	`backend/pipeline`
Parsing	`src/parsers/{windstream,peerless,att,spectrum,ocr}/*`	`backend/pipeline` pdfplumber / panda
Extraction	— (hand-coded inside parsers)	`backend/pipeline` `configs/carriers` `configs/carriers`
Merging	`src/merge/{carrier}.py` , `src/merge/source_resolver.py`	`backend/pipeline` `configs/carriers`

CONCERN	PREVIOUS (DIGITAL-DIRECTION)	NEW (PLATFORM/
		(merge_rules)
Validation	src/rules/qa_checklist.py , src/rules/confidence.py	backend/pipeline
Compliance	—	backend/pipeline
Config loader	src/config/carrier_config.py per-carrier YAML	backend/config_l
Persistence	backend/db.py (SQLite)	backend/models/o pgvector)
Frontend	frontend/src/pages/{Login,Dashboard,ProjectDetail,Checklist}.tsx	frontend/src/ (N Review, Analytics, B
Output	src/output/inventory.py (multi-sheet Excel)	Database-first; Exce